

Term Project – Developer Documentation

HandwerkerKasse – Developer Documentation

Dozent: Stephan Bökelmann sboekelmann@ep1.rub.de Technische Hochschule Georg Agricola, Bochum

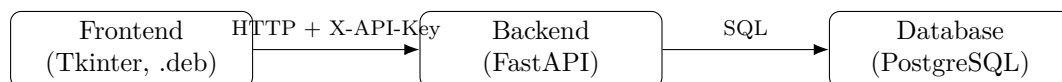
Author: Mohamed Yassine Hachimi

Module: Introduction to Database Management Systems · THGA Bochum

Audience: developers who want to run, test, extend, or deploy the system.

1 Architecture overview

HandwerkerKasse follows a three-tier architecture: a Tkinter desktop frontend communicates over HTTP with a FastAPI backend, which in turn talks to a PostgreSQL database. Write operations require a valid X-API-Key header; read operations are open.



Both the backend and the database run as containers, orchestrated with Docker Compose. The frontend is installed separately as a Debian package on the client machine.

2 Database schema

The database consists of five tables: `customers`, `jobs`, `materials`, `job_materials` and `invoices`. The schema follows the third normal form (3NF); every table stores one type of information, and relationships are expressed through foreign keys.

```
CREATE TABLE customers (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(150) NOT NULL,  
    email       VARCHAR(150),  
    phone       VARCHAR(50)  
);  
  
CREATE TABLE jobs (  
    id          SERIAL PRIMARY KEY,  
    customer_id INTEGER NOT NULL REFERENCES customers(id) ON DELETE CASCADE,  
    title       VARCHAR(200) NOT NULL,  
    status      VARCHAR(50) NOT NULL DEFAULT 'open'  
                CHECK (status IN ('open', 'in_progress', 'completed', 'cancelled')),  
    job_date    DATE NOT NULL DEFAULT CURRENT_DATE  
);
```

```

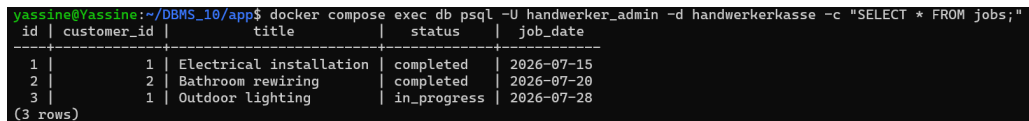
CREATE TABLE materials (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    purchase_price NUMERIC(10,2) NOT NULL CHECK (purchase_price >= 0)
);

CREATE TABLE job_materials (
    id SERIAL PRIMARY KEY,
    job_id INTEGER NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    material_id INTEGER NOT NULL REFERENCES materials(id) ON DELETE RESTRICT,
    quantity NUMERIC(10,2) NOT NULL CHECK (quantity > 0),
    UNIQUE (job_id, material_id)
);

CREATE TABLE invoices (
    id SERIAL PRIMARY KEY,
    job_id INTEGER NOT NULL UNIQUE REFERENCES jobs(id) ON DELETE CASCADE,
    amount NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    payment_status VARCHAR(50) NOT NULL DEFAULT 'unpaid'
    CHECK (payment_status IN ('unpaid','paid','overdue'))
);

```

Figure 1 shows the database populated with seed data, queried directly via `psql`. Figure 2 shows the PostgreSQL container running in a healthy state.



```

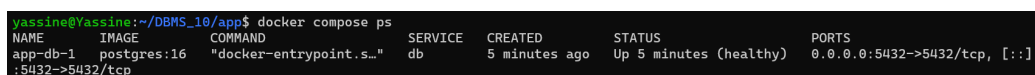
yassine@Yassine:~/DBMS_10/app$ docker compose exec db psql -U handwerker_admin -d handwerkerkasse -c "SELECT * FROM jobs;"

```

id	customer_id	title	status	job_date
1	1	Electrical installation	completed	2026-07-15
2	2	Bathroom rewiring	completed	2026-07-20
3	1	Outdoor lighting	in_progress	2026-07-28

(3 rows)

Abbildung 1: Seed data loaded into the jobs table



```

yassine@Yassine:~/DBMS_10/app$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
app-db-1	postgres:16	"docker-entrypoint.s..."	db	5 minutes ago	Up 5 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Abbildung 2: PostgreSQL container running and healthy

3 API reference

The FastAPI backend exposes the following endpoints. All write operations (POST, PUT, DELETE) require a valid `X-API-Key` header; read operations (GET) are open.

Method	Path	Purpose
GET	/customers	Returns all customers.
POST	/customers	Creates a new customer. Requires X-API-Key.
GET	/jobs	Returns all jobs.
POST	/jobs	Creates a new job. Requires X-API-Key.
PUT	/jobs/{id}	Updates an existing job. Requires X-API-Key.
DELETE	/jobs/{id}	Deletes an existing job. Requires X-API-Key.
GET	/materials	Returns all materials.
POST	/materials	Creates a new material. Requires X-API-Key.
POST	/job_materials	Assigns a material and quantity to a job. Requires X-API-Key.
GET	/invoices	Returns all invoices.
POST	/invoices	Creates a new invoice. Requires X-API-Key.
GET	/jobs/profit	Returns jobs with calculated profit values.

Figure 3 shows the interactive API documentation, automatically generated by FastAPI and available at /docs on the running server.



Abbildung 3: Automatically generated Swagger UI listing all endpoints

3.1 Profit calculation

The endpoint GET /jobs/profit is the core feature of the system. It joins jobs, invoices, job_materials and materials, aggregates the material cost per job, and returns:

$$\text{Profit} = \text{Invoice Amount} - \sum (\text{Quantity} \times \text{Purchase Price})$$

Figure 4 shows a live response from this endpoint, confirming the calculation works correctly against real data.

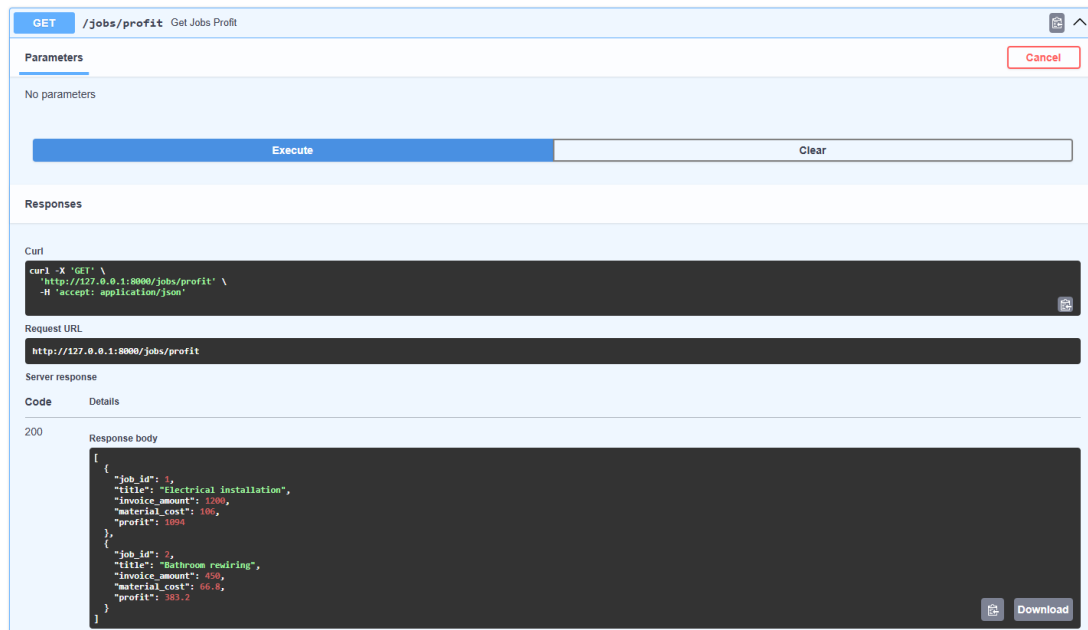


Abbildung 4: Live response of GET /jobs/profit

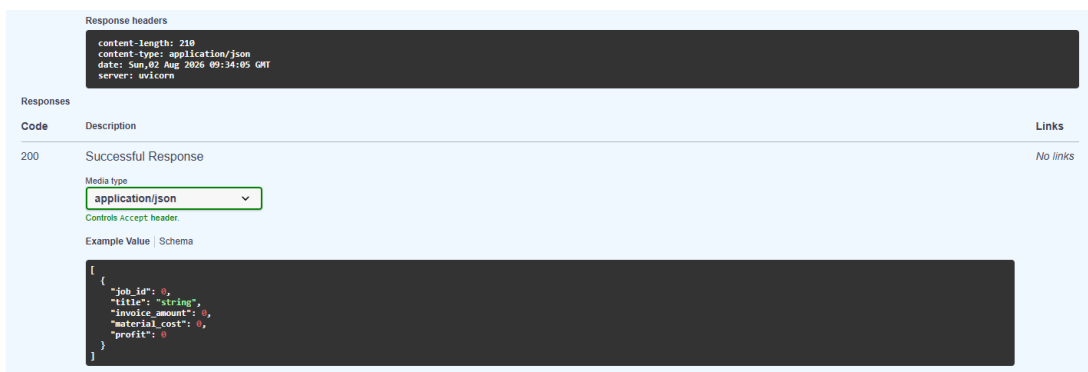


Abbildung 5: Live response of GET /jobs/profit (continued)

3.2 Authentication

Write endpoints reject requests without a valid `X-API-Key` with HTTP 401. Figure 6 shows a rejected request, and Figure 7 shows the same request succeeding once the correct key is provided.

POST /customers Create Customer

Parameters

Name	Description
x-api-key * required	x-api-key
string	(header)

Request body required application/json

Edit Value | Schema

```
{  "name": "string",  "email": "string",  "phone": "string"}
```

Please correct the following validation errors and try again.

- For 'x-api-key': Required field is not provided.

Abbildung 6: POST /customers rejected without a valid X-API-Key

x-api-key * required

string (header)

Request body required application/json

Edit Value | Schema

```
{  "name": "Test Kunde",  "email": "test@example.com",  "phone": "0123456789"}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \  "https://127.0.0.1:8080/customers" \  -H 'accept: application/json' \  -H 'x-api-key: change_me_super_secret_key' \  -H 'Content-Type: application/json' \  -d '{  "name": "Test Kunde",  "email": "test@example.com",  "phone": "0123456789"  }'
```

Abbildung 7: POST /customers succeeding with a valid X-API-Key



Abbildung 8: POST /customers response body showing the created customer

3.3 Reading data

Figures 9 and 10 show a full request/response cycle for GET /customers, confirming that seeded data is returned correctly as JSON.

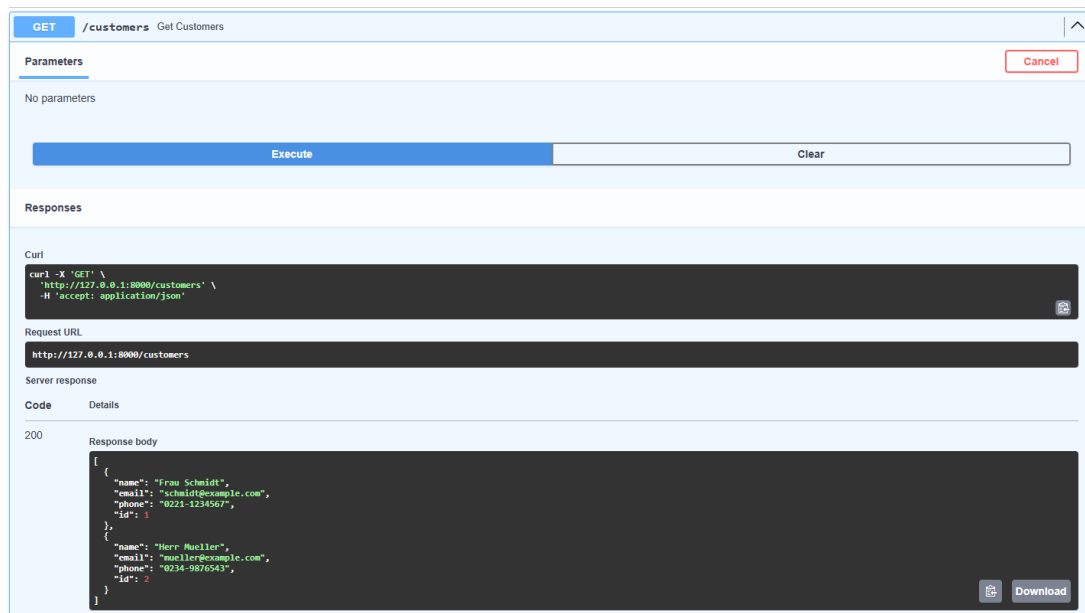


Abbildung 9: GET /customers request in Swagger UI

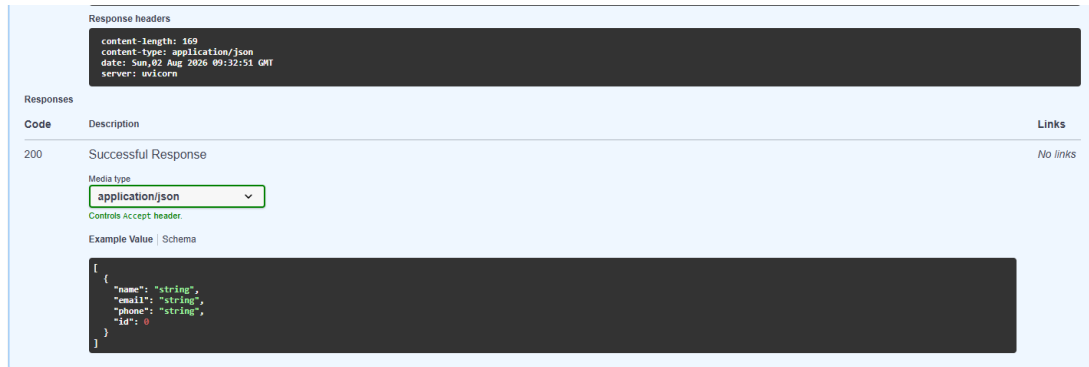


Abbildung 10: GET /customers response body

4 Local development setup

To run the backend locally without Docker:

```
cd app/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
```

The backend expects a .env file (in app/) with the following variables:

```
POSTGRES_USER=...
POSTGRES_PASSWORD=...
POSTGRES_DB=...
POSTGRES_PORT=5432
API_KEY=...
```

4.1 Project structure

```
app/
  docker-compose.yml
  .env
  db/
    schema.sql
    seed.sql
  backend/
    Dockerfile
    main.py
    database.py
    models.py
    schemas.py
    requirements.txt
    conftest.py
    test_api.py
  frontend/
    api.py
    ui.py
    connection_dialog.py
```

```

__main__.py
requirements.txt
packaging/
handwerkerkasse/
  DEBIAN/control
  opt/handwerkerkasse/main.py
  usr/bin/handwerkerkasse
  usr/share/applications/handwerkerkasse.desktop

```

5 Testing

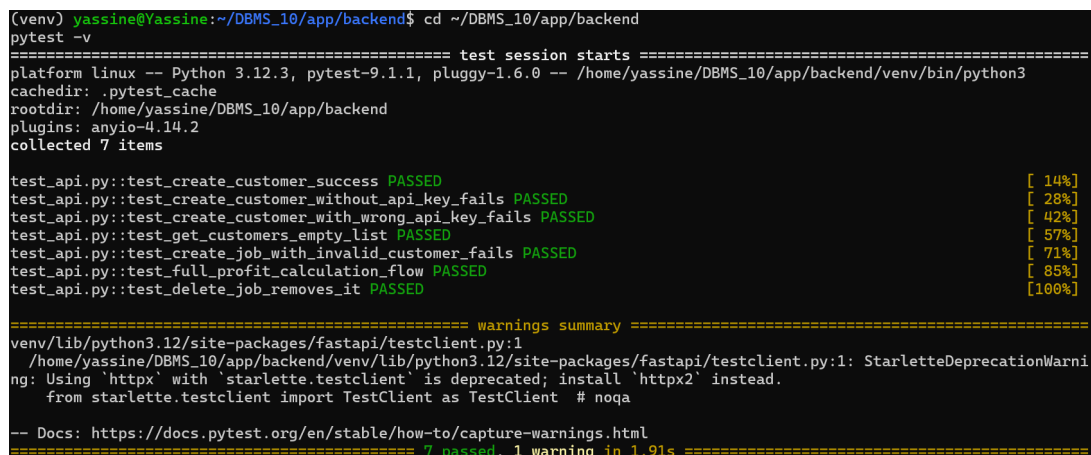
The backend is covered by an automated pytest suite (`test_api.py`), using FastAPI's `TestClient` against an isolated test database. Tests cover the happy path, authentication failures, invalid foreign keys, and a full end-to-end scenario verifying the profit calculation.

```

cd app/backend
source venv/bin/activate
pytest -v

```

Figure 11 shows a full test run, confirming that all acceptance criteria defined in the proposal are met.



```

(venv) yassine@Yassine:~/DBMS_10/app/backend$ cd ~/DBMS_10/app/backend
pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/yassine/DBMS_10/app/backend/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/yassine/DBMS_10/app/backend
plugins: anyio-4.14.2
collected 7 items

test_api.py::test_create_customer_success PASSED [ 14%]
test_api.py::test_create_customer_without_api_key_fails PASSED [ 28%]
test_api.py::test_create_customer_with_wrong_api_key_fails PASSED [ 42%]
test_api.py::test_get_customers_empty_list PASSED [ 57%]
test_api.py::test_create_job_with_invalid_customer_fails PASSED [ 71%]
test_api.py::test_full_profit_calculation_flow PASSED [ 85%]
test_api.py::test_delete_job_removes_it PASSED [100%]

===== warnings summary =====
venv/lib/python3.12/site-packages/fastapi/testclient.py:1
/home/yassine/DBMS_10/app/backend/venv/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarni
ng: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 7 passed, 1 warning in 1.91s =====

```

Abbildung 11: All automated tests passing

6 Deployment with Docker Compose

The backend and database are deployed together using Docker Compose. From the `app/` directory:

```
docker compose up -d --build
```

This builds the backend image, starts PostgreSQL, waits for its health check, then starts the backend container connected to it via the internal Docker network (service name `db`). Figure 12 shows both containers running.

```
(venv) yassine@yassine:~/DBMS_10/app$ docker compose ps
NAME                IMAGE             COMMAND                  SERVICE    CREATED         STATUS           PORTS
app-backend-1       app-backend       "uvicorn main:app --..." backend     10 seconds ago  Up 4 seconds    0.0.0.0:8000->8000/tcp, [
:]:8000->8000/tcp
app-db-1            postgres:16       "docker-entrypoint.s..." db         10 seconds ago  Up 9 seconds (healthy)  0.0.0.0:5432->5432/tcp, [
:]:5432->5432/tcp
[{"name":"Frau Schmidt", "email":"schmidt@example.com", "phone":"0221-1234567", "id":1}, {"name":"Herr Mueller", "email":"mueller@example.com", "phone":"0234-9876543", "id":2}, {"name":"Test Kunde", "email":"test@example.com", "phone":"0123456789", "id":3}]
BMS_10/app$ cd ~/DBMS_10
```

Abbildung 12: Database and backend containers running together

7 Building and installing the frontend package

The frontend is packaged as a Debian package for easy installation on client machines. The package structure is built manually and assembled with `dpkg-deb`:

```
cd packaging
dpkg-deb --build --root-owner-group handwerkerkasse
sudo apt install ./handwerkerkasse.deb
```

Once installed, the application can be launched from the applications menu or via the `handwerkerkasse` command. Figure 13 shows the installed application running.

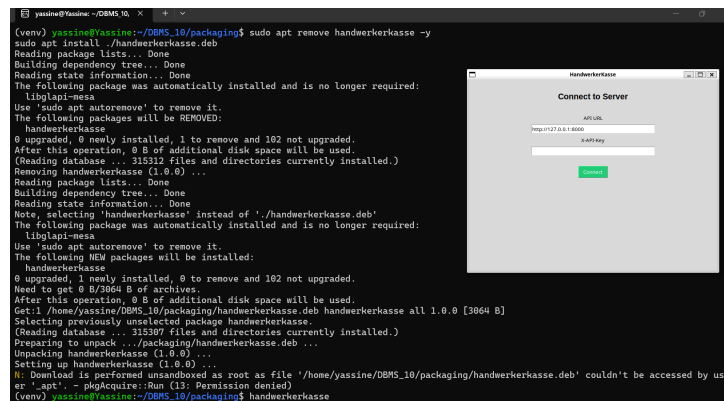


Abbildung 13: Installed .deb package running as a desktop application

8 Extending the system

The schema and API are designed to be extended without breaking existing functionality:

- New fields can be added to existing tables with `ALTER TABLE`, keeping foreign key relationships intact.
- New endpoints follow the same pattern: a Pydantic schema in `schemas.py`, a SQLAlchemy model in `models.py` if needed, and a route in `main.py` using the `verify_api_key` dependency for write access.
- Possible extensions include inventory tracking for materials, multi-user support, or a dedicated endpoint for updating and deleting job-material assignments.

9 Design decisions and how to modify them

This section explains the reasoning behind key implementation choices, which parts of the system are optional or interchangeable, and how a developer would go about changing them.

9.1 ORM vs. raw SQL (optional choice)

The backend uses **SQLAlchemy** as an ORM instead of writing raw SQL queries directly with **psycopg2**. This was a deliberate but optional choice:

- Each database table is represented as a Python class in `models.py` (e.g. `class Job(Base)`), with columns defined as class attributes and relationships declared explicitly (e.g. `relationship("Customer", back_populates="jobs")`).
- This makes queries readable in Python (`db.query(models.Job).filter(...)`) instead of embedding SQL strings throughout the code, and it automatically handles foreign key joins through the declared relationships.
- **To switch to raw SQL:** replace the SQLAlchemy query calls in `main.py` with **psycopg2** cursor calls (already installed as a dependency, since SQLAlchemy uses it internally as the database driver). The database schema itself (`db/schema.sql`) would not need to change, since it is plain SQL independent of the ORM.

9.2 Pydantic schemas (required, but extensible)

Every endpoint uses a Pydantic model from `schemas.py` to validate incoming data (e.g. `CustomerCreate`) and to shape outgoing responses (e.g. `CustomerOut`). This is not optional – FastAPI relies on these schemas for automatic request validation and for generating the Swagger documentation seen in Figure 3.

To add a new field (e.g. a `notes` field on jobs): add the column in `db/schema.sql`, add the attribute to the `Job` class in `models.py`, and add it to `JobCreate` and `JobOut` in `schemas.py`. No changes to `main.py` are needed, since the existing endpoints already forward all fields via `model_dump()`.

9.3 Authentication (optional to extend)

Authentication is currently a single shared **X-API-Key**, checked by the `verify_api_key` dependency in `main.py`, applied per-endpoint via `dependencies=[Depends(verify_api_key)]`. This was a deliberate simplification for the project's scope (no user accounts, as agreed in the proposal).

To extend this to per-user authentication: a `users` table could be added, `verify_api_key` replaced with a function that validates a JWT or session token instead of a static string, and the dependency reused unchanged on all write endpoints, since the endpoint logic itself does not need to know how authentication is implemented.

9.4 Frontend framework (optional choice)

The frontend uses Python's built-in **tkinter** library rather than a more feature-rich framework such as PyQt. This was chosen for simplicity and easy `.deb` packaging, since **tkinter** ships with Python and requires no additional system libraries beyond `python3-tk`.

To replace the frontend entirely: any client capable of sending HTTP requests with an **X-API-Key** header can be used instead, since the frontend and backend are fully decoupled and communicate only over the documented REST API. A web frontend, a mobile app, or a different desktop framework could all be substituted without changing the backend.

9.5 Docker Compose networking (fixed requirement)

Unlike the previous points, the way the backend locates the database is **not** optional: the backend connects to the database using the Docker Compose service name (`db`) as the hostname rather than `localhost`, since containers on the same Compose network resolve each other by service name. This is controlled by the `DB_HOST` environment variable, set to `db` in `docker-compose.yml` and defaulting to `localhost` for local development outside Docker (see `database.py`).

9.6 Summary of extension points

- **Easy to change:** frontend framework, authentication mechanism, ORM vs. raw SQL.
- **Easy to extend without breaking anything:** new fields, new tables, new endpoints following the existing pattern.
- **Fixed by the deployment architecture:** the Docker Compose service name as the database host, and the REST/JSON contract between frontend and backend.

10 Reflection questions

This section answers the kind of conceptual questions a developer new to the project might ask, in the style used throughout this module.

Question: Why separate the API client logic from the UI code?

In the frontend, all HTTP communication happens through simple function-style calls inside each screen class (e.g. `requests.post(...)` in `JobFormScreen.save_job()`), rather than being scattered across the whole application. This keeps each screen responsible only for collecting input and displaying results, while the actual request format (headers, JSON body, error handling) stays consistent. If the API changed its authentication method, only these call sites would need updating, not the widget layout.

Question: Why does the backend use dependency injection (`Depends(get_db)`) instead of a global database connection?

Each request gets its own database session via `Depends(get_db)`, which is created fresh and closed automatically after the request finishes (see the `finally: db.close()` block in `database.py`). A single global connection shared across requests would risk one request's uncommitted changes leaking into another, especially under concurrent load, which FastAPI supports by default.

Question: Why is the profit calculation done in the database query rather than in Python after fetching the data?

`GET /jobs/profit` performs the JOIN and aggregation directly in SQL (via SQLAlchemy's query builder), rather than fetching all jobs, materials and invoices separately and calculating the sum in a Python loop. This is both more efficient (the database engine is optimized for aggregation) and more correct, since the calculation logic lives in one place rather than being duplicated if another endpoint needed the same figure.

Question: What would happen if the X-API-Key check were removed from a write endpoint by mistake?

Any client could create, modify or delete data without authorization. This is exactly what the automated test `test_create_customer_without_api_key_fails` in `test_api.py` guards

against: it asserts that such a request is rejected with a 401 status. If a future change accidentally removed the `Depends(verify_api_key)` dependency from an endpoint, this test would fail immediately, catching the regression before deployment.

Question: Why does the backend fail to start if it cannot reach the database?

SQLAlchemy's `create_engine()` does not connect immediately; the actual connection attempt happens on the first query. This is why, during development, a misconfigured `DB_HOST` produced no error until the first API call was made – the container appeared to start successfully, but every request failed with a connection error. This was resolved by explicitly setting `DB_HOST=db` in `docker-compose.yml`, matching the database service's name on the Docker network.

Question: What is the benefit of testing against a separate test database instead of the development database?

The pytest suite creates and drops all tables in a dedicated `handwerkerkasse_test` database for every test function (see `conftest.py`). This guarantees that tests start from a clean, predictable state and never depend on or corrupt the seed data used for manual testing and demonstration.